

Shipyard: A Multi-Leader Consensus Protocol with Auto-Balanced Leadership of the Sharded Keyspace

ILLINOIS INSTITUTE
OF TECHNOLOGY



Xincheng Yang, Kyle Hale*
xyang76@hawk.illinoistech.edu, kyle.hale@oregonstate.edu

(I am seeking postdoc positions!)

• This work is accepted at IPDPS 2026



Introduction

Typical SMR(state machine replication) protocols (e.g., Paxos, Raft, Zab) use a **single-leader** design

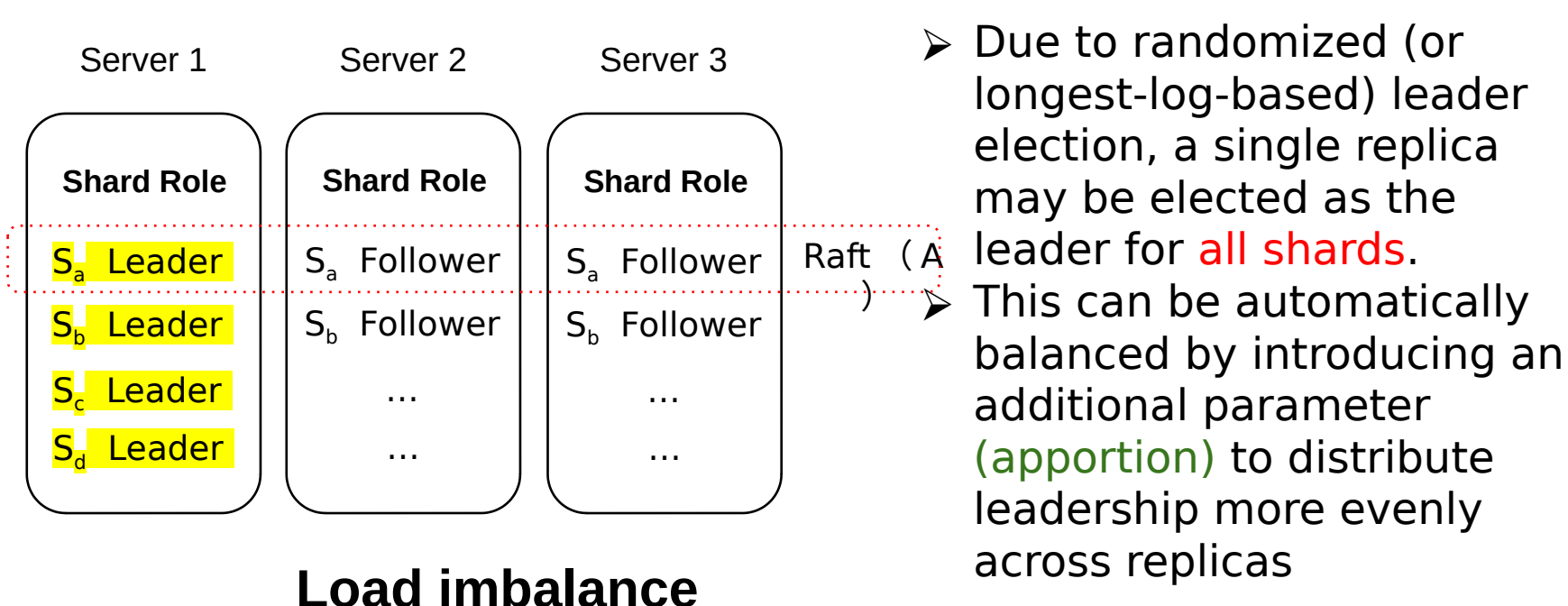
- **Performance bottleneck** under high load.

Keyspace sharding improves scalability via multiple independent shards, however, those shards may not be balanced

- Load imbalance

Challenges

In sharding-based SMR systems, workload cannot be **automatically balanced** through leadership management



System Model

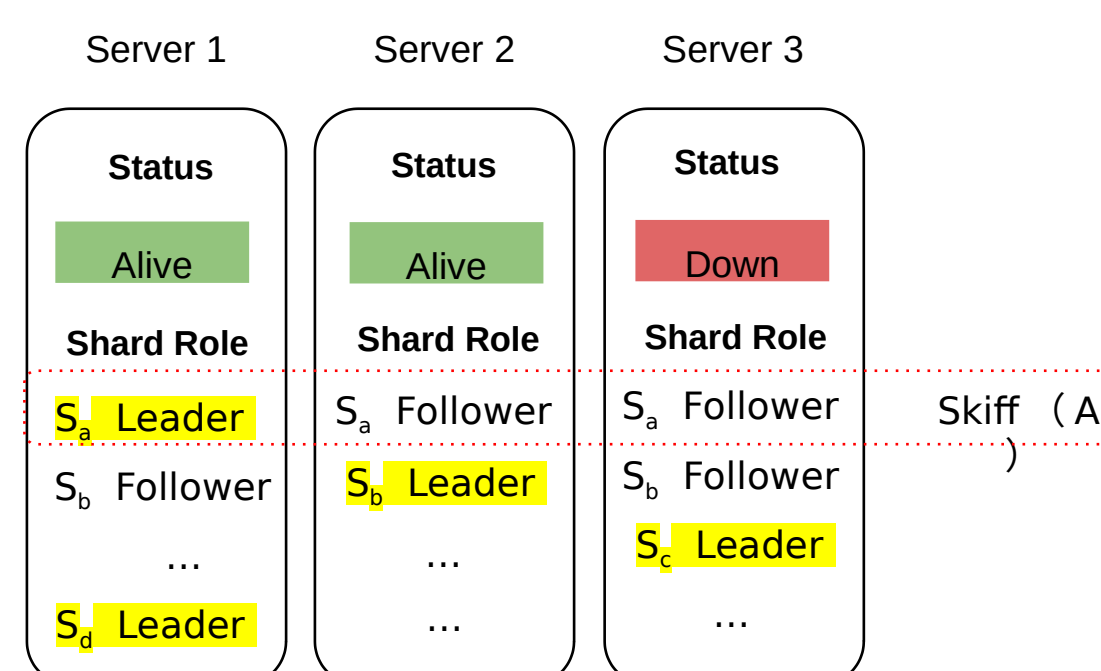
We address on **sharding-based replication systems**:

- Data is partitioned into multiple shards. Within each shard, data is replicated across replicas
- Multiple clients concurrently submit requests to any replica.
- Replicas can fail and later recover.
- A quorum of nodes ensures consistency. A majority of quorum maintains liveness and reaches consensus.
- All nodes are trusted (no Byzantine faults)

Shipyard Overview

Shipyard ensures linearizability. Two main parts:

- Skiff (per shard): workload-aware consensus
- Shipyard (global): multi-shard coordination



Traditional SMR



Paxos



Raft

(n = term / v = log entry)

● Two phases

- **A monotonically increase integer number N**
Used to order / distinguish proposal rounds
- **A chosen value V**
The value to be agreed upon for replication

Key Idea (Apportion)

Extending the traditional monotonic **integer**

$N \rightarrow \langle \text{epoch}, \text{apportion} \rangle \text{ pair}$

- Epoch: preserves global ordering under failures
- Apportion: captures load distribution
 - Derived from CPU usage, request rate, or number of leader shards
 - **Embedded** load information into the consistency protocol
 - Higher apportion $\rightarrow$ higher chance to become a leader during election



Acknowledgements

This project is made possible by support from the NSF via grants CNS-2533773, CNS-2239757, and CNS-2310422.

Skiff (Three-Phases)

01 VoteAndGather(Edited)

Vote: Votes for the leader.

Gather: Gather logs from peers to maintain correctness(ensure log is majority replicated).

02 AppendEntries

The leader orders and replicates command.

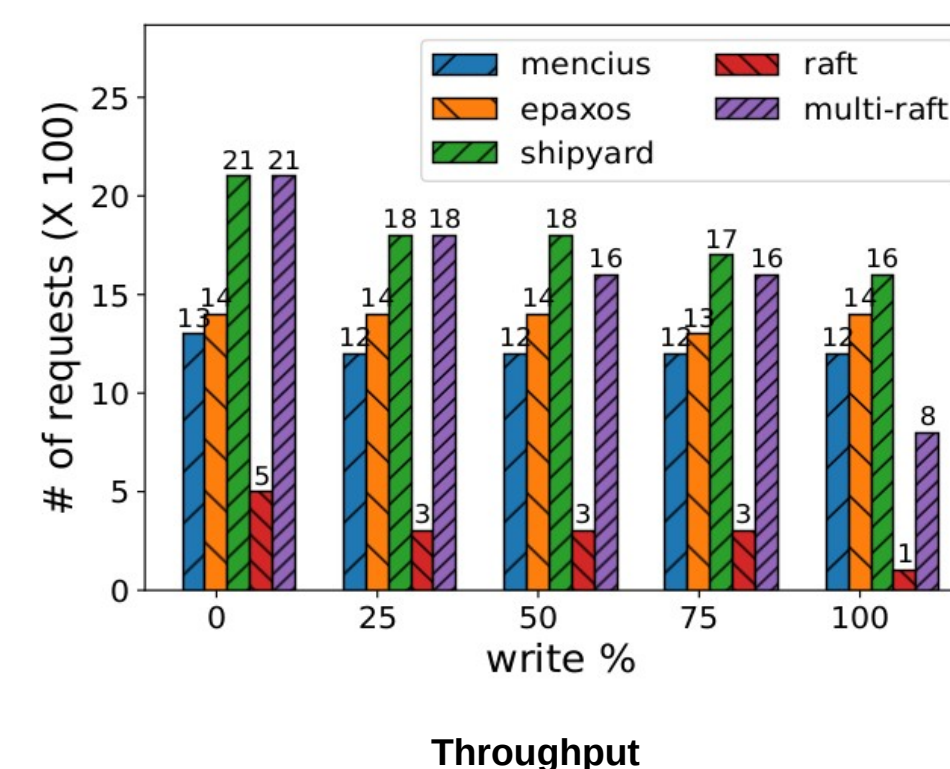
03 Balance(New)

If the follower detect the leader is not balanced based on its apportion comparison, start a balance RPC.

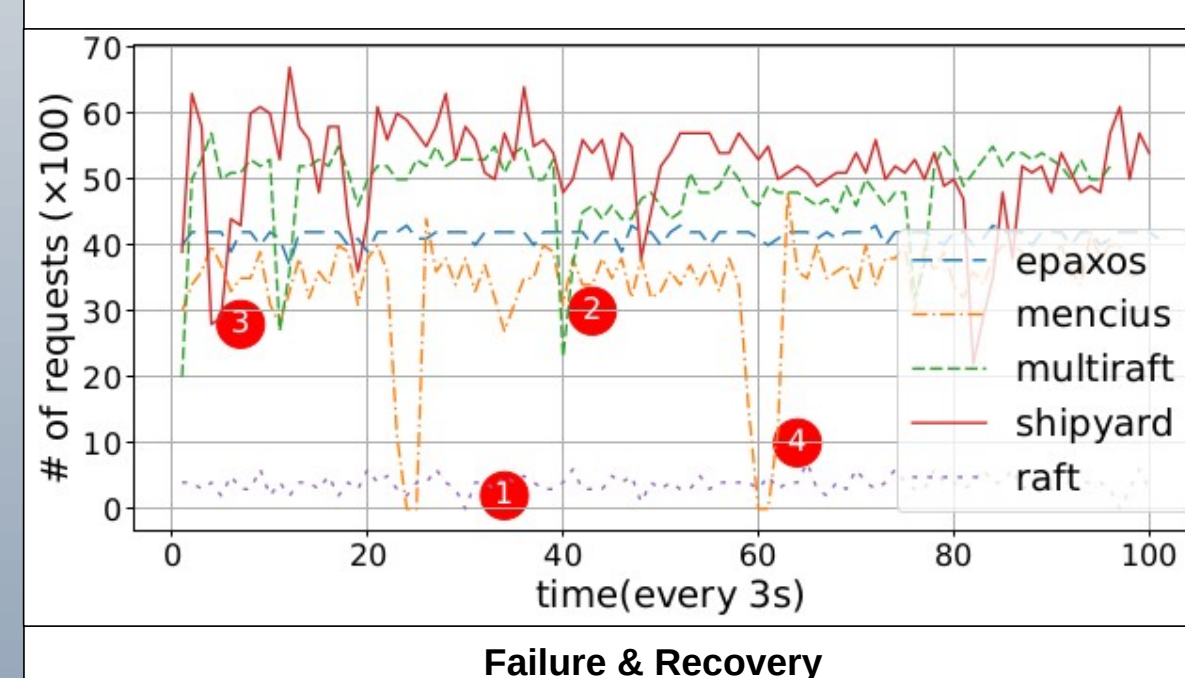
Shipyard Protocol

- Manages all sharded skiff instances
- Unified heartbeat
 - Shipyard periodically send heartbeats among replicas
- Decouples fault tolerance from ordering
 - Each shard (Skiff protocol) handles command ordering independently
 - Fault tolerance delegated to Shipyard

Evaluation (Amazon EC2)



- Shipyard achieves 1.3x~1.5x better performance than EPaxos and Mencius
- Raft (single-leader) is the slowest due to the single-leader bottleneck
- Multi-Raft assigns leaders randomly and does not prioritize load balancing



- Multi-leader improves availability vs. single-leader Raft
- Shipyard trades longer recovery for more stable performance via rebalancing
- EPaxos is resilient, Mencius suffers longer unavailability, Multi-Raft is sensitive to leader placement

Future Work

Sharding may weaken system-wide safety guarantees

- Investigate BFT(Byzantine-fault-tolerant) cross-shard malicious behaviors
- Develop runtime detection and accountability mechanisms

Try Out Our Code

Shipyard (Go lang, based on EPaxos):

<https://github.com/xyang76/Shipyard>



References

- [1] L. Lamport, "Paxos Made Simple," SIGACT News, 2001
- [2] D. Ongaro et al., "In Search of an Understandable Consensus Algorithm," USENIX ATC '14
- [3] Moraru et al., "There is More Consensus in Egalitarian Parliaments," SOSP '13
- [4] Mao et al., "Mencius: Building Efficient Replicated State Machines for WANs," OSDI '08